

Laboratorul 2

Managementul proceselor & comunicarea intre procese via semnale

1. Procesele si managementul acestora

- **Proces** = instanta a unui program care ruleaza;
- Sistemul de operare tine evidenta proceselor prin intermediul unui ID de cinci cifre, denumit **PID**; fiecare proces din sistem are un **PID** unic;
- Rularea unui proces poate fi facuta in doua moduri:
 - **foreground** - fiecare proces pornit de utilizator ruleaza implicit in acest mod; input-ul se primeste de la tastatura, iar output-ul se afiseaza pe ecran. Daca un proces ruleaza in foreground, in timpul executiei acestuia nu pot fi rulate alte comenzi, anume nu pot fi pornite alte procese (prompt-ul devine din nou disponibil dupa terminarea executiei procesului curent);

Ex: `ls a*.doc` (comanda bash ls va porni in executie o instanta a executabilului ls si va afisa rezultatul executiei in terminal);

- **background** - permite executia altor comenzi de la linia de comanda - nu trebuie asteptata terminarea executiei primului proces pentru a executa altul; daca un proces din background necesita input de la tastatura, acesta intra intr-o stare de oprire pana cand este mutat in foreground si primeste datele necesare (anume nu influenteaza in mod blocant sesiunea din terminal);

Ex: `ls a*.doc &` (pornirea unui proces in background);

- Listarea proceselor se face utilizand comanda `ps -aux` (-a = ofera informatii despre toti utilizatorii, -u = informatii aditionale, -x = ofera informatii despre procesele fara terminal)
- Inchiderea unui proces:
 - In **foreground** - `CTRL + C`
 - In **background** - obtinem *PID-ul* cu ajutorul comenzii `ps`, iar apoi folosim comanda `kill` pentru a intrerupe procesul din executie

\$ps -f

UID	PID	PPID	C	STIME	TTY	TIME	CMD
georgiana	6738	3662	0	10:23:03	pts/6	0:00	first_one
georgiana	6739	3662	0	10:22:54	pts/6	0:00	second_one
georgiana	3662	3657	0	08:10:53	pts/6	0:00	-ksh
georgiana	6892	3662	4	10:51:50	pts/6	0:00	ps -f

\$kill 6738

- **Starile** unui proces:

- *Start* - starea initiala cand un proces este mai intai inceput/creat;
- *Ready* - procesul este pregatit de executie si asteapta sa ii fie asignat un procesor de catre sistemul de operare, astfel incat sa poata rula; ele pot ajunge in starea aceasta dupa Start sau in timpul rularii (sunt intrerupte de catre scheduler pentru a aloca CPU si altor procese);
- *Running* - dupa asignarea la un procesor; se executa instructiunile;
- *Waiting* - asteapta o resursa (input de la utilizator, un fisier care este blocat de alt proces etc.)
- *Finished* - dupa finalizarea executiei sau atunci cand este oprit de sistemul de operare; asteapta sa fie eliminat din memoria principala;
- *Zombie* - este identificat de un "Z" in dreptul lui in tabela de procese obtinuta la utilizarea comenzii ps; procesul si-a completat executia, nu mai are resurse alocate dar inca exista in tabelul de procese.
- *Orphan* - daca procesul parinte este omorat inainte ca procesul copil sa se fi incheiat, acesta este "adoptat" de procesul "init" (primeste PPID-ul acestuia);

- **Suspendarea** unui proces: CTRL + Z

- Primitive de interes pentru manipularea proceselor: `fork()`, `exec()`, `getpid()`, `getppid()`, `sleep()`, `usleep()`;

2. Exemplu privind primitiva fork()

```
/* fork.c
   Exemplu de executie concurenta a doua procese parinte si copil, folosind
   primitiva
   fork.
   Autor: Lenuta Alboaie <adria@infoiasi.ro> (c)2009
*/
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

main()
{
    pid_t pid;
    int rv;

    switch(pid=fork()) {
        case -1:
            perror("fork");
            exit(1);

        case 0:
            printf(" PC: Proces Copil!\n");
            printf(" PC: Am PID-ul %d\n", getpid());
            printf(" PC: PID-ul parintelui este %d\n", getppid());
            printf(" PC: Da-mi un status de iesire: ");
            scanf(" %d", &rv);
            printf(" PC: Am plecat!\n");
            exit(rv);

        default:
            printf("PP: Proces Parinte!\n");
            printf("PP: Am PID-ul %d\n", getpid());
            printf("PP: Am copil cu pidul %d\n", pid);
            printf("PP: Astept ca PC sa apeleze exit()...\n");
            wait(&rv);
            printf("PP: Statusul de iesire al PC este: %d\n",
WEXITSTATUS(rv));
            printf("PP: Am plecat!\n");
    }
}
```

3. Exemplu folosind primitivele `fork()`, `execlp()`, `wait()`

```
/* exec.c

Executia prin program a comenzii 'ls -a -l', folosind execlp()

Autor: Sabin-Corneliu Buraga <busaco@infoiasi.ro> (c)1998-2001
Versiunea: 1.0 (ultima actualizare: 04 iunie 2001)
*/
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>

int
main ()
{
    pid_t pid;                /* PID-ul procesului copil */
    int status;               /* starea de terminare a procesului copil */

    printf ("Vom executa comanda...\n");

    if ((pid = fork ()) < 0)
    {
        perror ("fork()");
        exit (1);
    }
    else if (pid)              /* parinte */
    {
        if (wait (&status) < 0)
        {
            perror ("wait()");
        }
        printf ("Comanda a fost executata.\n");
        exit (0);
    }
    else                      /* fiu */
    {
        /* vom folosi execlp() */
        execlp ("ls",
                /* comanda de executat (se va cauta in directoarele din PATH)
                "ls",          /* argv[0] */
                "-a",          /* argv[1] */
                "-l",          /* argv[2] */
                NULL);
        /* daca ajungem aici inseamna ca nu s-a putut executa */
        printf ("Eroare de executie!\n");
        exit (1);
    }
}
```

4. Detalii si exemple privind semnalele

- **Semnale** = intreruperi software generate in urma producerii unui eveniment si transmise unui proces de catre sistemul de operare;
- Cauzele aparitiei unui semnal sunt evenimentele generate de hardware, evenimentele generate de sistemul de operare, si evenimentele provocate de utilizator;
- Lista de semnale suportate de sistem poate fi obtinuta prin rularea comenzii *kill -l*;
- Fiecare semnal al sistemului de operare are asociat o actiune implicita care trebuie executata la primirea semnalului de catre program;
- Exemple de semnale uzuale:

Signal Name	Signal Number	Description
SIGHUP	1	Hang up detected on controlling terminal or death of controlling process
SIGINT	2	Issued if the user sends an interrupt signal (Ctrl + C)
SIGQUIT	3	Issued if the user sends a quit signal (Ctrl + D)
SIGFPE	8	Issued if an illegal mathematical operation is attempted
SIGKILL	9	If a process gets this signal it must quit immediately and will not perform any clean-up operations
SIGALRM	14	Alarm clock signal (used for timers)
SIGTERM	15	Software termination signal (sent by kill by default)

- Primitive utile in manipularea semnalelor: **signal**, **sigaction**, **kill**, **raise**, **sigprocmask**(pentru blocarea de semnale), **sigpending** (pentru aflarea de semnale din coda de semnale), **sigsuspend**, **sigfillset**, **sigdelset**, **sigsuspend**;

```

/* sig.c

    Afiseaza continutul fisierului "/etc/services" la aparitia
    semnalului SIGUSR2

    Autor: Sabin-Corneliu Buraga <busaco@infoiasi.ro> (c)1998-2001
    Ultima actualizare: 10 iulie 2001
*/
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/types.h>

void sighandler (int sig)          /* functia de tratare a semnalului
*/
{
    pid_t pid;

    if ((pid = fork ()) < 0)      /* executam 'less' ca proces copil */
    {
        perror ("fork()");
        exit (1);
    }
    if (!pid)                    /* copil */
    {
        execl ("/usr/bin/less", "less", "/etc/services", NULL);
        perror ("exec()");
        exit (1);
    }
    else                          /* parinte */
    {
        if (wait (NULL) < 0)
            perror ("wait()");
    }
}

int main ()                      /* programul principal */
{
    /* atasam functia de tratare la semnalul SIGUSR2 */
    if (signal (SIGUSR2, sighandler) == SIG_ERR)
    {
        perror ("signal()");
        return 1;
    }
    /* asteptam aparitia unui semnal */
    pause ();
    return 0;
}

```